

Smart Eco-Fi Vendo System

Engineering Design Blueprint & Technical Specification: Dual-Controller Reverse Vending Machine for Automated Wi-Fi Hotspot Time Provisioning.

- Host: Orange Pi (Armbian / OpenWrt)
- Controller: ESP32 DevKit V1
- Sensing: 4-Tier Anti-Cheat
- Protocol: Structured JSON UART

1. System Architecture & Distributed Computing Model

The machine utilizes a **Distributed Hybrid Architecture** to decouple real-time hardware sensing from Linux network routing:

Hardware Sub-Controller (ESP32)

- **Sub-ms I/O Interrupts:** Optical drop detection (<5ms).
- **Anti-Fraud Gatekeeping:** Inductive metal rejection & weight validation.
- **Actuator Control:** MG996R drop hatch and 12V reject solenoid.
- **Deterministic JSON Frames:** Emits credit events over serial UART.

Core Gateway & Server (Orange Pi)

- **Network Gateway:** DHCP, DNS redirect, and bandwidth QoS.
- **Captive Portal Engine:** Responsive user landing interface.
- **Session Accounting:** SQLite-backed MAC/IP credit table.
- **Dynamic Firewall:** Real-time Linux ipset / iptables authorization.



2. Mechanical Intake & Anti-Fraud Pipeline

1. **Stage 1 (Intake Scan):** Door switch wakes the system. Load cell confirms object weight is within standard PET range (10g–65g). Inductive proximity sensor checks for metal/tin cans.
2. **Stage 2 (Retention & Drop):** If valid, MG996R servo rotates 90° opening the internal hatch. Top IR (#1) and bottom IR (#2) trigger sequentially to confirm full descent without string pullback.
3. **Stage 2-Alt (Rejection):** If metal is detected or weight is out of bounds, the 12V solenoid ejects the item to the return tray with an audible warning.
4. **Stage 3 (Credit Transmission):** ESP32 sends { "event": "CREDIT_ADD", "bottles": 1 } to the Orange Pi.
5. **Stage 4 (Provisioning):** Orange Pi updates ipset to permit unrestricted internet access (+15 min/bottle) for the active user session.

3. Complete Bill of Materials (BOM)

Computing, Logic & Networking

Component	Specification	Qty	Engineering Purpose
Main Host SBC	Orange Pi Zero 3 (2GB) / 3B	1 pc	Linux routing, captive portal web server, iptables firewall.
Microcontroller	ESP32 DevKit V1 (38-Pin)	1 pc	Real-time anti-cheat sensor reading, motor control, UART packets.
MicroSD Card	SanDisk Extreme Pro 32GB (A1/V30)	1 pc	High-endurance storage for OS, SQLite session DB, and logs.
High-Gain AP	TP-Link / Comfast Outdoor AP	1 pc	Long-range Wi-Fi broadcasting and 50+ concurrent clients.
Interconnects	Cat6 UTP (0.5m) + USB-A Bridge (0.3m)	1 set	Low-latency Ethernet and isolated USB-to-UART comms link.

Sensors, Actuation & UI

Component	Specification	Qty	Engineering Purpose
Drop IR Sensors	E18-D80NK Adjustable IR	2 pcs	Dual optical beam-break verifying complete drop trajectory.
Weight Sensor	1kg–5kg Load Cell + HX711 Module	1 set	Mass validation (10g–65g empty PET bottle threshold).
Metal Sensor	LJ12A3-4-Z/BX Inductive Proximity	1 pc	Detects aluminum/tin cans to prevent non-plastic jamming.
Bin Full Sensor	JSN-SR04T Waterproof Ultrasonic	1 pc	Measures storage fill depth; halts intake when bin is full.
Gate Servo	MG996R High-Torque Metal Gear	1 pc	Actuates drop hatch trapdoor inside the validation chute.
Reject Solenoid	12V Push-Pull Open Frame Solenoid	1 pc	Ejects rejected items to the exterior return tray.
Displays	20x4 I2C LCD + 0.96" I2C OLED	2 pcs	System status, deposit prompts, and voucher PIN display.
Feedback / Inputs	5V Buzzer + 3x LEDs + Metal Button	1 set	Audio chirps, status indicators, and session finish trigger.

Power Distribution & Cabinet

Component	Specification	Qty	Engineering Purpose
Main SMPS	12V 5A (60W) Switching PSU	1 unit	Converts AC grid power to primary 12V DC system bus.
Buck Regulators	XL4015 (5A) + LM2596 (3A)	2 pcs	Isolated 5.1V logic rail and independent 5.0V motor rail.
Inlet & Cooling	IEC C14 Socket (Fused) + 12V Fan	1 set	Mains AC protection and electronics compartment cooling.
Cabinet & Storage	Plywood/Sheet Metal + 60L Bin	1 set	Secure housing cabinet with service access door and tote.

4. Electrical Interfacing & Pin Mapping

Module	ESP32 / OPi Pin	Voltage	Circuit Topology Note
Top IR (E18-D80NK #1)	GPIO 18	5V Rail / 3.3V Logic	Direct signal to GPIO with internal INPUT_PULLUP.
Bottom IR (E18-D80NK #2)	GPIO 19	5V Rail / 3.3V Logic	Direct signal to GPIO with internal INPUT_PULLUP.
Metal Proximity (LJ12A3)	GPIO 23	12V Rail / 3.3V Logic	NPN output with 10kΩ/10kΩ resistor voltage divider.
Load Cell (HX711)	DT: GPIO 4 SCK: GPIO 5	3.3V Rail	2-wire synchronous serial data and clock.
Bin Ultrasonic (JSN-SR04T)	Trig: GPIO 14 Echo: 12	5V Rail / 3.3V Logic	Echo line level-shifted to 3.3V logic.
Hatch Servo (MG996R)	GPIO 13 (PWM)	5V Motor Buck Rail	PWM signal from ESP32; VCC isolated to LM2596 rail.

Module	ESP32 / OPi Pin	Voltage	Circuit Topology Note
Reject Solenoid (12V)	GPIO 32	12V Rail	Driven via IRLZ44N MOSFET with 1N4007 flyback diode.
20x4 LCD & OLED Displays	SDA: GPIO 21 SCL: GPIO 22	5V & 3.3V Rails	Shared I2C bus (LCD at 0x27, OLED at 0x3C).
Serial Host Bridge	ESP32 USB → OPi USB	5V USB Standard	UART0 link accessed via /dev/ttyUSB0 at 115200 baud.

5. ESP32 Controller Firmware (C++)

```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <ESP32Servo.h>
#include "HX711.h"

#define PIN_IR_TOP      18
#define PIN_IR_BOTTOM  19
#define PIN_PROX_METAL  23
#define PIN_HX711_DT    4
#define PIN_HX711_SCK   5
#define PIN_ULTRASONIC_TRIG 14
#define PIN_ULTRASONIC_ECHO 12
#define PIN_DOOR_SWITCH 27
#define PIN_SERVO_GATE  13
#define PIN_SOLENOID_REJ 32
#define PIN_BUZZER       33
#define PIN_LED_GREEN    25
#define PIN_LED_RED      26
#define PIN_BTN_FINISH   34

const float MIN_PET_WEIGHT = 10.0;
const float MAX_PET_WEIGHT = 65.0;
const int BIN_FULL_THRESHOLD_CM = 15;

LiquidCrystal_I2C lcd(0x27, 20, 4);
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
Servo hatchServo;
HX711 scale;

volatile bool topIrTriggered = false;
volatile bool bottomIrTriggered = false;
int currentSessionBottles = 0;
bool isBinFull = false;

void IRAM_ATTR isrTopIr() { topIrTriggered = true; }
void IRAM_ATTR isrBottomIr() { bottomIrTriggered = true; }

void buzz(int durationMs, int pulses = 1) {
  for(int i = 0; i < pulses; i++) {
    digitalWrite(PIN_BUZZER, HIGH);
    delay(durationMs);
    digitalWrite(PIN_BUZZER, LOW);
    if(pulses > 1) delay(80);
  }
}

int getBinDistanceCm() {
  digitalWrite(PIN_ULTRASONIC_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_ULTRASONIC_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_ULTRASONIC_TRIG, LOW);
  long duration = pulseIn(PIN_ULTRASONIC_ECHO, HIGH, 30000);
  if (duration == 0) return 999;
  return duration * 0.034 / 2;
}

void triggerRejection(const char* reason) {
  digitalWrite(PIN_LED_RED, HIGH);
  digitalWrite(PIN_LED_GREEN, LOW);
  lcd.setCursor(0, 1); lcd.print("STATUS: REJECTED!  ");
  lcd.setCursor(0, 2); lcd.print(reason);
  buzz(600, 1);
  digitalWrite(PIN_SOLENOID_REJ, HIGH);
  delay(500);
  digitalWrite(PIN_SOLENOID_REJ, LOW);
  delay(2000);
  digitalWrite(PIN_LED_RED, LOW);
}
```

```

}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_IR_TOP, INPUT_PULLUP);
  pinMode(PIN_IR_BOTTOM, INPUT_PULLUP);
  pinMode(PIN_PROX_METAL, INPUT_PULLUP);
  pinMode(PIN_DOOR_SWITCH, INPUT_PULLUP);
  pinMode(PIN_BTN_FINISH, INPUT_PULLUP);
  pinMode(PIN_ULTRASONIC_TRIG, OUTPUT);
  pinMode(PIN_ULTRASONIC_ECHO, INPUT);
  pinMode(PIN_SOLENOID_REJ, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_LED_GREEN, OUTPUT);
  pinMode(PIN_LED_RED, OUTPUT);

  attachInterrupt(digitalPinToInterrupt(PIN_IR_TOP), isrTopIr, FALLING);
  attachInterrupt(digitalPinToInterrupt(PIN_IR_BOTTOM), isrBottomIr, FALLING);

  hatchServo.attach(PIN_SERVO_GATE);
  hatchServo.write(0);

  scale.begin(PIN_HX711_DT, PIN_HX711_SCK);
  scale.set_scale(420.0);
  scale.tare();

  lcd.init();
  lcd.backlight();
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setTextColor(WHITE);
  oled.setCursor(0, 10);
  oled.println("Eco-Fi Vendo Ready");
  oled.display();

  lcd.setCursor(0, 0); lcd.print("=== ECO-FI VENDO ===");
  lcd.setCursor(0, 1); lcd.print("Ready for Deposit ");
  lcd.setCursor(0, 2); lcd.print("Rate: 1 Bottle = 15m");
  lcd.setCursor(0, 3); lcd.print("Session Bottles: 0 ");
}

void loop() {
  int binDistance = getBinDistanceCm();
  if (binDistance < BIN_FULL_THRESHOLD_CM && binDistance > 0) {
    if (!isBinFull) {
      isBinFull = true;
      lcd.setCursor(0, 1); lcd.print("STATUS: STORAGE FULL");
      lcd.setCursor(0, 2); lcd.print("Empty Bin Required ");
      digitalWrite(PIN_LED_RED, HIGH);
    }
    delay(1000);
    return;
  } else {
    isBinFull = false;
  }

  float weight = scale.get_units(3);

  if (weight >= MIN_PET_WEIGHT) {
    digitalWrite(PIN_LED_GREEN, HIGH);
    delay(350);

    if (digitalRead(PIN_PROX_METAL) == LOW) {
      triggerRejection("Tin/Can Detected ");
      scale.tare();
      return;
    }

    if (weight > MAX_PET_WEIGHT) {
      triggerRejection("Liquid / Overweight ");
      scale.tare();
      return;
    }

    topIrTriggered = false;
    bottomIrTriggered = false;
    lcd.setCursor(0, 1); lcd.print("STATUS: VERIFIED OK ");
    hatchServo.write(90);

    unsigned long gateOpenTime = millis();
    bool passedDrop = false;

    while (millis() - gateOpenTime < 3000) {
      if (topIrTriggered && bottomIrTriggered) {
        passedDrop = true;

```

```

        break;
    }
    delay(20);
}

hatchServo.write(0);

if (passedDrop) {
    currentSessionBottles++;
    buzz(120, 2);
    digitalWrite(PIN_LED_GREEN, HIGH);

    Serial.print("{\"event\":\"CREDIT_ADD\",\"bottles\":1,\"sessionTotal\":");
    Serial.print(currentSessionBottles);
    Serial.println("}");

    lcd.setCursor(0, 1); lcd.print("STATUS: BOTTLE SAVED");
    lcd.setCursor(0, 3); lcd.print("Session Bottles: ");
    lcd.print(currentSessionBottles);
    delay(1200);
} else {
    triggerRejection("Drop Optical Timeout");
}
scale.tare();
} else {
    digitalWrite(PIN_LED_GREEN, LOW);
    digitalWrite(PIN_LED_RED, LOW);
}

if (digitalRead(PIN_BTN_FINISH) == LOW && currentSessionBottles > 0) {
    Serial.print("{\"event\":\"SESSION_FINISH\",\"totalBottles\":");
    Serial.print(currentSessionBottles);
    Serial.println("}");

    buzz(300, 1);
    lcd.setCursor(0, 1); lcd.print("SESSION FINALIZED! ");
    lcd.setCursor(0, 2); lcd.print("Enjoy Your Internet ");
    delay(3000);

    currentSessionBottles = 0;
    lcd.setCursor(0, 1); lcd.print("Ready for Deposit ");
    lcd.setCursor(0, 2); lcd.print("Rate: 1 Bottle = 15m");
    lcd.setCursor(0, 3); lcd.print("Session Bottles: 0 ");
}
delay(100);
}
}

```

6. Orange Pi Hotspot Service & Network Daemon (Python)

```

#!/usr/bin/env python3
import serial, json, time, subprocess, logging, sqlite3

SERIAL_PORT = "/dev/ttyUSB0"
BAUD_RATE = 115200
MINUTES_PER_BOTTLE = 15
DB_PATH = "/opt/ecofi/vendo_sessions.db"

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute('''CREATE TABLE IF NOT EXISTS sessions (
                mac_address TEXT PRIMARY KEY,
                ip_address TEXT,
                expiry_time DATETIME,
                bottles_deposited INTEGER)''')
    conn.commit()
    conn.close()

def setup_firewall_ipset():
    try:
        subprocess.run("ipset create ecofi_auth hash:ip timeout 86400 -exist", shell=True, check=True)
        subprocess.run("iptables -t nat -C PREROUTING -m set --match-set ecofi_auth dst -j ACCEPT 2>/dev/null || "
                        "iptables -t nat -I PREROUTING 1 -m set --match-set ecofi_auth dst -j ACCEPT", shell=True, check=True)
    except subprocess.CalledProcessError as e:
        logging.error(f"Firewall setup error: {e}")

def grant_access(ip_address: str, duration_minutes: int):
    timeout_sec = duration_minutes * 60
    cmd = f"ipset add ecofi_auth {ip_address} timeout {timeout_sec} -exist"
    try:
        subprocess.run(cmd, shell=True, check=True)
        logging.info(f"Granted {duration_minutes} mins to IP: {ip_address}")
    
```

```

except subprocess.CalledProcessError as err:
    logging.error(f"IPSet execution failed: {err}")

def get_latest_portal_ip() -> str:
    try:
        with open("/tmp/current_active_client.txt", "r") as f:
            return f.read().strip()
    except FileNotFoundError:
        return "192.168.45.100"

def process_packet(payload: dict):
    event = payload.get("event")
    if event == "CREDIT_ADD":
        bottles = payload.get("bottles", 1)
        added_minutes = bottles * MINUTES_PER_BOTTLE
        target_ip = get_latest_portal_ip()
        logging.info(f"Credit event: {bottles} bottle(s) -> +{added_minutes} mins to {target_ip}")
        grant_access(target_ip, added_minutes)
    elif event == "SESSION_FINISH":
        total = payload.get("totalBottles", 0)
        logging.info(f"Session closed. Total bottles: {total}")

def main():
    init_db()
    setup_firewall_ipset()
    logging.info("Eco-Fi Host Service Initialized. Connecting to ESP32...")
    while True:
        try:
            with serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2) as ser:
                logging.info(f"Serial open on {SERIAL_PORT}")
                while True:
                    raw_line = ser.readline().decode('utf-8', errors='ignore').strip()
                    if not raw_line: continue
                    try:
                        data = json.loads(raw_line)
                        process_packet(data)
                    except json.JSONDecodeError:
                        logging.warning(f"Malformed serial frame: {raw_line}")
        except (serial.SerialException, FileNotFoundError):
            logging.warning("ESP32 serial disconnected. Retrying in 3s...")
            time.sleep(3)

if __name__ == "__main__":
    main()

```

7. Systemd Service Configuration & Calibration Protocol

```

[Unit]
Description=Eco-Fi Reverse Vending Machine Hotspot Daemon
After=network.target network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=/opt/ecofi
ExecStart=/usr/bin/python3 /opt/ecofi/daemon.py
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target

```

- HX711 Calibration:** Place a 20g calibration weight on the intake cradle. Adjust scale .set_scale(value) in firmware until reading reads exactly 20.00g.
- Inductive Trimming:** Adjust the potentiometer on the back of the LJ12A3-4-Z so that metal cans trigger detection at 4mm-6mm, while the plastic cradle produces no false trigger.
- IR Distance Setup:** Turn the multi-turn screw on both E18-D80NK sensors until the LED indicates triggering at the chute inner wall boundary (~15cm).
- Power Rail Check:** Confirm the LM2596 motor rail stays $\geq 4.85V$ under full servo load.
- Launch Service:**

```
sudo systemctl daemon-reload && sudo systemctl enable --now ecofi.service
```